

Introdução

O PANDORA é uma plataforma web interna para a gestão de locações e manutenções dos equipamentos próprios da Mazzi. Esta modelagem apresenta propostas de aplicação dos padrões criacionais Singleton, Builder, Factory Method e Abstract Factory, estudados em Linguagem de Programação III.

Os diagramas detalham responsabilidades de criação de objetos e complementam o modelo de classes do domínio. As classes de apoio e as interfaces representam decisões preliminares de projeto, sujeitas à validação durante a implementação. Para cada padrão, são apresentados o contexto de uso, sua estrutura e os impactos esperados.

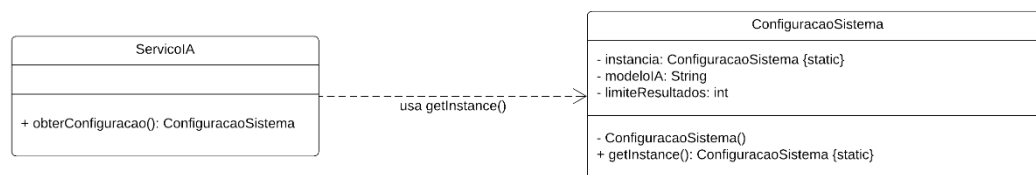
1. Singleton

O Singleton pode centralizar configurações compartilhadas do serviço de inteligência artificial, como o modelo utilizado e o limite de resultados. A classe `ConfiguracaoSistema` mantém a instância estática e oferece `getInstance()`, que cria o objeto na primeira chamada e o reutiliza nas seguintes. O construtor privado restringe a criação direta; `ServicoIA` consulta essa instância.

A proposta evita carregar configurações repetidamente, mas introduz estado global e pode dificultar testes isolados.

A unicidade vale por processo, não para todos os servidores. A inicialização deve ser segura em chamadas concorrentes, e o objeto não deve armazenar dados mutáveis de usuários. Se as configurações nativas do framework atenderem à necessidade, uma classe Singleton adicional será dispensável.

PANDORA - 1. Singleton



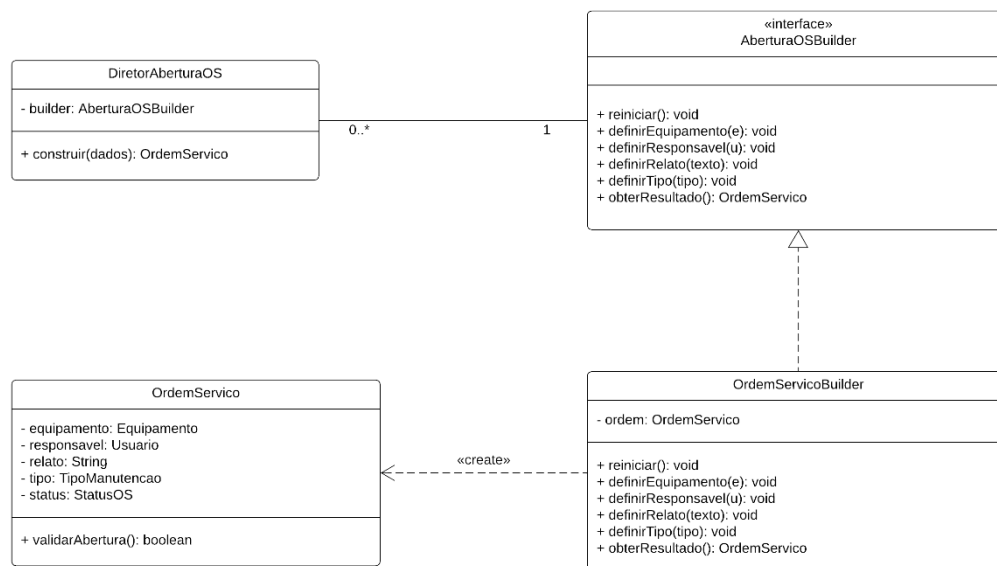
{Uma única instância por processo; inicialização protegida contra concorrência.}
`getInstance()` cria na primeira chamada e reutiliza nas seguintes.

2. Builder

O Builder organiza a preparação de uma ordem de serviço em etapas. DiretorAberturaOS coordena a interface AberturaOSBuilder, implementada por OrdemServicoBuilder. O construtor concreto inicia uma nova OrdemServico, define equipamento, responsável, relato e tipo de manutenção e devolve o resultado após validar os dados necessários à abertura.

A separação facilita o reaproveitamento das etapas e evita construtores com muitos parâmetros. Atividades executadas, testes finais e liberação técnica continuam pertencendo ao ciclo posterior da manutenção. O builder deve ser reiniciado a cada construção e não pode compartilhar uma OS em andamento entre requisições. O custo é a criação de classes adicionais, justificável quando a montagem possuir etapas ou variações relevantes.

PANDORA - 2. Builder



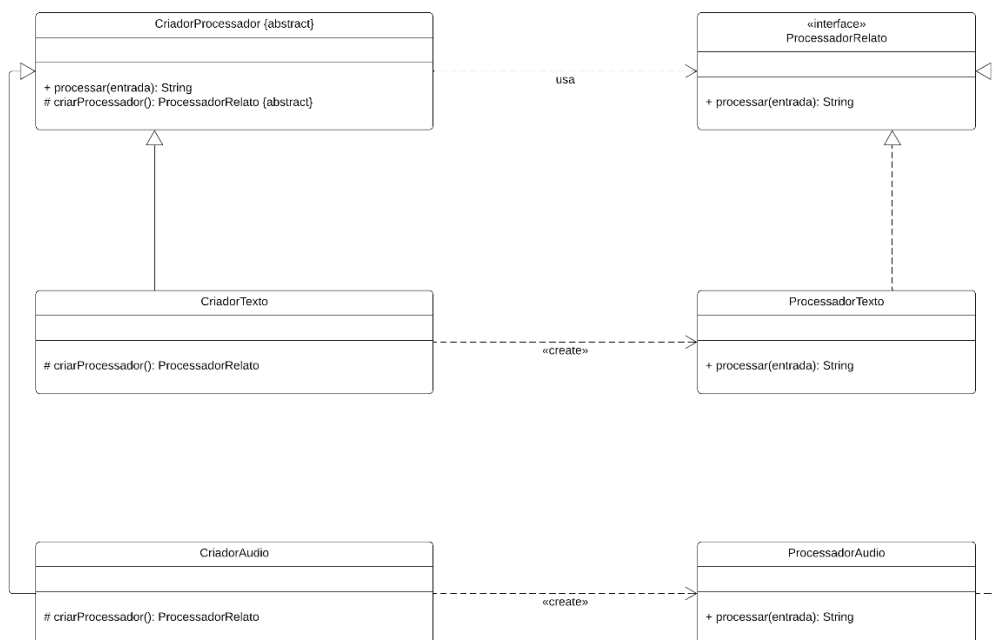
O diretor coordena as etapas; o builder concreto monta uma nova OS e valida a abertura. Atividades executadas e teste final pertencem às etapas posteriores da manutenção.

3. Factory Method

O Factory Method pode selecionar a criação dos processadores de relatos técnicos recebidos em texto ou áudio. CriadorProcessador define o fluxo comum processar() e o método de fábrica abstrato criarProcessador(). CriadorTexto e CriadorAudio sobrescrevem esse método para criar ProcessadorTexto e ProcessadorAudio, respectivamente. Ambos os produtos implementam ProcessadorRelato e retornam texto normalizado; o processador de áudio inclui a transcrição.

O fluxo comum depende da interface do produto, permitindo acrescentar processadores sem alterar sua lógica. A configuração da aplicação escolhe o criador adequado à entrada. Essa organização facilita testes e manutenção, mas aumenta o número de classes. A transcrição deve permanecer sujeita à revisão humana antes de integrar o histórico técnico; o padrão não garante sua precisão.

PANDORA - 3. Factory Method



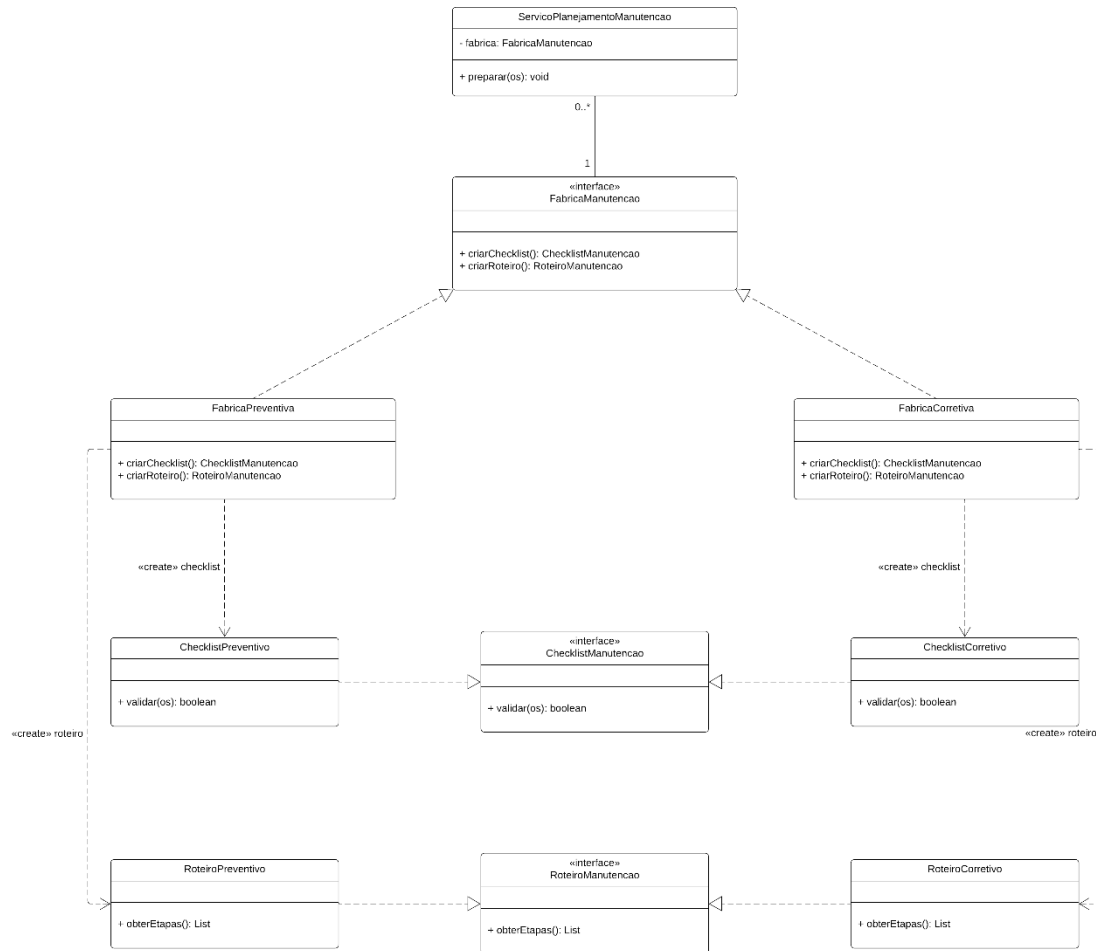
processar() chama criarProcessador() e delega o tratamento ao produto retornado.
 Texto: normaliza o relato. Áudio: transcreve e normaliza; a saída segue para revisão humana.

4. Abstract Factory

O Abstract Factory é proposto para criar famílias compatíveis de objetos de apoio à manutenção preventiva ou corretiva. `FabricaManutencao` declara `criarChecklist()` e `criarRoteiro()`. `FabricaPreventiva` produz `ChecklistPreventivo` e `RoteiroPreventivo`; `FabricaCorretiva` produz `ChecklistCorretivo` e `RoteiroCorretivo`. Os produtos realizam as interfaces `ChecklistManutencao` e `RoteiroManutencao`.

`ServicoPlanejamentoManutencao` recebe a fábrica correspondente ao tipo da OS e utiliza os produtos por suas interfaces. Assim, cada planejamento reúne checklist e roteiro da mesma modalidade, sem duplicar a entidade `OrdemServico`. O padrão facilita trocar a família completa, mas exige novas classes e alterações nas fábricas quando surge outro tipo de produto. Sua adoção depende de comportamentos realmente distintos entre as modalidades; diferenças apenas nos dados podem ser atendidas por configuração.

PANDORA - 4. Abstract Factory



Cada fábrica produz checklist e roteiro da mesma modalidade de manutenção.
Proposta aplicável se as modalidades exigirem comportamentos distintos; a entidade OrdemServico permanece única.